Flask Routing and Views Lab (CodeGrade)

[(https://github.com/learn-co-curriculum/python-p4-routing-and-views-lab)](https://github.com/learn-co-curriculum/python-p4-routing-and-views-lab) [(https://github.com/learn-co-curriculum/python-p4-routing-and-views-lab/issues/new)](https://github.com/learn-co-curriculum/python-p4-routing-and-views-lab/issues/new)

Learning Goals

- Build and run a Flask application on your computer.
- Manipulate and test the structure of a request object.

Key Vocab

- **Web Framework**: software that is designed to support the development of web applications. Web frameworks provide built-in tools for generating web servers, turning Python objects into HTML, and more.
- **Extension**: a package or module that adds functionality to a Flask application that it does not have by default.
- **Request**: an attempt by one machine to contact another over the internet.
- **Client**: an application or machine that accesses services being provided by a server through the internet.
- **Web Server**: a combination of software and hardware that uses Hypertext Transfer Protocol (HTTP) and other protocols to respond to requests made over the internet.
- **Web Server Gateway Interface (WSGI)**: an interface between web servers and applications.
- **Template Engine**: software that takes in strings with tokenized values, replacing the tokens with their values as output in a web browser.

Instructions

This is a **test-driven lab**. Run `pipenv install` to create your virtual environment and `pipenv shell` to enter the virtual environment. Then enter the `server/` directory and run `pytest -x` to run your tests. Use these instructions and `pytest` 's error messages to complete your work in the `server/` directory. If you prefer working in a Flask environment to running your application as a script, remember to configure it with the following commands from the `server/` directory:

```
$ export FLASK_APP=app.py
$ export FLASK_RUN_PORT=5555
```

Instructions begin here:

- Design a Flask application that carries out basic Python functions using routing and views.
- Your `index()` view should be routed to at the base URL with `/` . It should Contain an `h1` element that contains the title of this application, "Python Operations with Flask Routing and Views".
- A `print_string` view should take one parameter, a string. It should print the string in the console *and* display it in the web browser. Its URL should be of the format `/print/parameter` .
- A `count()` view should take one parameter, an integer. It should display all numbers in the `range` of that parameter on separate lines. Its URL should be of the format `/count/parameter` .
- A `math()` view should take three parameters: `num1` , `operation` , and `num2` . It must perform the appropriate operation on the two numbers in the order that they are presented. The included operations should be: `+` , `-` , `*` , `div` (`/` would change the URL path), and `%` . Its URL should be of the format `/math/<num1>/<operation>/<num2>` .

Once all of your tests are passing, commit and push your work using `git` to submit.

Resources

- [A Minimal Application - Flask (https://flask.palletsprojects.com/en/2.2.x/quickstart/#a-minimal-application)](https://flask.palletsprojects.com/en/2.2.x/quickstart/#a-minimal-application)
- [Routing - Flask (https://flask.palletsprojects.com/en/2.2.x/quickstart/#routing)](https://flask.palletsprojects.com/en/2.2.x/quickstart/#routing)
- [Chapter 2. Basic Application Structure - Flask Web Development, 2nd Edition (https://learning.oreilly.com/library/view/flask-web-development/9781491991725/ch02.html#idm140583868985008)](https://learning.oreilly.com/library/view/flask-web-development/9781491991725/ch02.html#idm140583868985008)

This tool needs to be loaded in a new browser window

Load Flask Routing and Views Lab (CodeGrade) in a new window